Studio Nx

Backend(FastAPI/Python) 43 — GET /api/v1/profile-images/{name}

2026-10-07

Backend(FastAPI/Python) 43 — `GET /api/v1/profile-images/{name}`

항목	내용
파일	`studionx/backend/app/api/v1/users.py` — `images_router` · `read_profile_image()` / `app/api/v1/router.py` (한 줄)
작업	P1-08 회원 정보 — 프로필 사진 파일 보내기 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	42 `POST · DELETE /users/me/profile-image` · 44 `check current password()` (P1-09)

1. 하는 일

`<img src="/api/v1/profile-images/3f9c….jpg">` 가 부르는 주소. 사진 **파일 자체**를 보낸다. 로그인은 필요 없다.

요청	응답
올린 사진의 주소 (토큰 없이)	200 그림 바이트 · `Content-Type: image/jpeg` (png · webp 도 맞게) · `X-Content-Type-Options: nosniff` · `Cache-Control: public, max-age=86400`
없는 이름 · 모양이 아닌 이름	404 `NOT_FOUND` "Image not found."
지운 사진 · 탈퇴한 회원의 사진	404 (같음)
`..%2F..%2F.env`	404 — `/` 로 풀려 이 주소와 맞지도 않음 (앱 공통 404)
템플릿의 `/api/v1/users/85/profile-image`	404 — **없는 주소**

2. 언제 쓰나

쓰는 곳	
화면의 `<img src>`	userInfo.html · myPage.html 의 프로필 사진 (P1-10 · P1-17), 협업 멤버 사진 (Phase 6)
주소는 어디서	36 응답의 `profile_image_url` (35 가 만듦) — 화면은 받은 주소를 그대로 넣기만

사용자 결정 반영 (35 §2)

템플릿은 `GET /users/{user_id}/profile-image` 였다. **회원 번호로 여는 공개 주소를 만들지 않는다**(탈퇴 추적 방지 ③)는 결정에 따라 `/profile-images/{무작위 이름}` 으로 바꿨다. 이 주소는 `/users` 아래가 아니어서 라우터를 하나 더 두었다.

바꾼 곳	내용
`users.py`	`images_router = APIRouter()` 와 그 위의 라우터 함수
`router.py`	`api_router.include_router(users.images_router, prefix="/profile-images", tags=["users"])`
`tests/test_user_routes.py`	연결 확인 시험을 새 주소로, 옛 주소는 **없음**을 확인

3. 코드

import (파일 맨 위 — 41 과 같음, 추가 없음). 이 함수는 `FileResponse` 와 40 `profile_image_file` 을 쓴다.

라우터 · 상수 (이번에 추가)

```
# 사진 파일 — /api/v1/profile-images/{name} (회원 번호가 아니라 무작위 파일 이름, 2026-10-07)
# /users 아래가 아니라서 라우터를 따로 둠 → router.py 에서 prefix="/profile-images" 로 붙임
images_router = APIRouter()

# 확장자 → 응답 종류 (PHOTO_TYPES 를 뒤집음: ".jpg" → "image/jpeg")
PHOTO_MEDIA_TYPES = {ext: media for media, ext in user_service.PHOTO_TYPES.items()}
```

함수

```
@images_router.get(
    "/{name}",
    response_class=FileResponse,
    responses={404: {"model": ErrorResponse}},
)
def read_profile_image(name: str, db: Db) -> FileResponse:
    path = user_service.profile_image_file(db, name)
    return FileResponse(
        path,
        media_type=PHOTO_MEDIA_TYPES[path.suffix],
        headers={"Cache-Control": "public, max-age=86400", "X-Content-Type-Options": "nosniff"},
    )
```

문법 풀이

식	뜻
`{ext: media for media, ext in ….items()}`	**사전 만들기 식** — `PHOTO_TYPES`(`"image/jpeg": ".jpg"`)의 키와 값을 뒤집은 사전
`"/{name}"` · `name: str`	주소의 마지막 부분이 `name` 매개변수로 들어옴
`FileResponse(path, …)`	파일을 조금씩 읽어 보냄 (통째로 메모리에 올리지 않음)
`path.suffix`	확장자 `".jpg"`
`response_class=FileResponse`	문서(`/docs`)에 "파일 응답" 으로 표시

4. 흐름

1. 40 으로 파일을 찾는다 — 못 보여 주면 거기서 404.
2. 확장자로 종류를 정하고, 캐시 · `nosniff` 헤더를 붙여 파일을 보낸다.

5. 왜 이렇게

선택	이유
로그인 확인 없음	`<img>` 는 `Authorization` 헤더를 붙일 수 없다. 대신 주소가 무작위 32자라 이름을 모르면 열 수 없다 (2^{128} 가지)

선택	이유
종류를 **확장자로** 직접 정함	38 이 내용을 보고 확장자를 붙였으니 확장자 = 실제 종류. 운영체제의 종류 표에 기대지 않는다 (Windows 는 `.webp` 를 모를 수 있음)
`X-Content-Type-Options: nosniff`	브라우저가 내용을 보고 "사실은 HTML 이네" 하고 바꿔 해석하지 않게 — 38 의 내용 검사와 이중 잠금
캐시 1일 (`max-age=86400`, 템플릿 제안)	같은 사진을 매번 받지 않는다. 사진을 바꾸면 **이름이 바뀌어** 새로 받는다. 더 길게(1년) 할 수도 있지만, 지운 사진 · 탈퇴한 회원의 사진이 다른 사람 브라우저에 오래 남지 않게 하루로
`public`	사진은 이름만 알면 누구나 보는 자료 — 공유 캐시(CDN)에 둬도 같다
라우터를 `users.py` 안에	회원 사진은 회원 정보의 일부. 파일을 새로 만들면 템플릿 머리말 · 안내가 갈라진다

6. 시험 (`tests/test_user_routes.py` 에 8개 추가 · 모두 26개, 실제 앱 + MySQL + 임시 폴더)

시험	확인한 것
jpg · png · webp 올린 뒤 응답 주소로 열기 (토큰 없이)	200, 바이트가 같음, `Content-Type` 이 각각 맞음, `nosniff`, 캐시 헤더
없는 이름 · `..%2F..%2F.env` · 모양 아님	404 `NOT_FOUND`
지운 사진 · 탈퇴 회원의 사진	404
옛 주소 `/users/{번호}/profile-image`	열리지 않음
연결 확인 (DB 없이)	`/api/v1/profile-images/{name}` 있음, 옛 주소 없음

처음 1개 실패: `%2F` 는 `/` 로 풀려 사진 주소와 맞지 않아 이 함수까지 오지 않고 앱 공통 404("Not found.")가 났다. 안전한 결과라 그 경우는 상태 · 코드만 확인하도록 시험을 고쳤다. 전체 시험 314개 통과, `ruff` 검사 통과. 진짜 `storage/` 는 비어 있음.